



Operations Research

Publication details, including instructions for authors and subscription information:
<http://pubsonline.informs.org>

Rescheduling for New Orders

Nicholas G. Hall, Chris N. Potts,

To cite this article:

Nicholas G. Hall, Chris N. Potts, (2004) Rescheduling for New Orders. Operations Research 52(3):440-453. <https://doi.org/10.1287/opre.1030.0101>

Full terms and conditions of use: <https://pubsonline.informs.org/Publications/Librarians-Portal/PubsOnLine-Terms-and-Conditions>

This article may be used only for the purposes of research, teaching, and/or private study. Commercial use or systematic downloading (by robots or other automatic processes) is prohibited without explicit Publisher approval, unless otherwise noted. For more information, contact permissions@informs.org.

The Publisher does not warrant or guarantee the article's accuracy, completeness, merchantability, fitness for a particular purpose, or non-infringement. Descriptions of, or references to, products or publications, or inclusion of an advertisement in this article, neither constitutes nor implies a guarantee, endorsement, or support of claims made of that product, publication, or service.

© 2004 INFORMS

Please scroll down for article—it is on subsequent pages



With 12,500 members from nearly 90 countries, INFORMS is the largest international association of operations research (O.R.) and analytics professionals and students. INFORMS provides unique networking and learning opportunities for individual professionals, and organizations of all types and sizes, to better understand and use O.R. and analytics tools and methods to transform strategic visions and achieve better outcomes.

For more information on INFORMS, its publications, membership, or meetings visit <http://www.informs.org>

Rescheduling for New Orders

Nicholas G. Hall

Department of Management Sciences, Fisher College of Business, The Ohio State University,
Columbus, Ohio 43210-1144, hall.33@osu.edu

Chris N. Potts

School of Mathematics, University of Southampton, Southampton, United Kingdom SO17 1BJ, c.n.potts@maths.soton.ac.uk

This paper considers scheduling problems where a set of original jobs has already been scheduled to minimize some cost objective, when a new set of jobs arrives and creates a disruption. The decision maker needs to insert the new jobs into the existing schedule without excessively disrupting it. Two classes of models are considered. First, we minimize the scheduling cost of all the jobs, subject to a limit on the disruption caused to the original schedule, where this disruption is measured in various ways. In the second class, a total cost objective, which includes both the original cost measure and the cost of disruption, is minimized. For both classes and various costs based on classical scheduling objectives, and for almost all problems, we provide either an efficient algorithm or a proof that such an algorithm is unlikely to exist. We also show how to extend both classes of models to deal with multiple disruptions in the form of repeated arrivals of new jobs. Our work refocuses the extensive literature on scheduling problems towards issues of rescheduling, which are important because of the frequency with which disruptions occur in manufacturing practice.

Subject classifications: production/scheduling; sequencing, deterministic, single machine.

Area of review: Manufacturing, Service, and Supply Chain Operations.

History: Received August 2002; revision received February 2003; accepted April 2003.

1. Introduction

It frequently happens in manufacturing practice that a *disruption* creates a need for *rescheduling*. Examples of common disruptions include: the arrival of new orders, order cancellations, changes in order priority, processing delays, changes in release dates, machine breakdowns, and the unavailability of raw materials, personnel, or tools. Rescheduling involves adjusting a previously planned, possibly optimal, schedule to account for a disruption. Despite the extensive literature on scheduling problems, there is relatively little research on rescheduling.

This paper considers the issue of how to reschedule jobs in a manufacturing environment that experiences a disruption. We assume that a set of original jobs has been scheduled optimally to minimize some natural cost objective. However, the processing of most of those jobs has not begun. This situation arises wherever schedules are planned in advance of their start date, in practice typically several weeks earlier. Based on this schedule, promises have been made to customers, and resource allocations over time have been determined. Then, either before or during the execution of the schedule, a set of new jobs arrives unexpectedly. It is important to minimize the cost of all the jobs with respect to the objective used in the original schedule. However, doing so will disrupt the original schedule, thereby inconveniencing the customers and creating havoc with the resource allocations. We develop models that address the trade-off between the scheduling cost of all the jobs and the degree of this disruption.

We model the amount of disruption both as a constraint and as part of an objective. In some situations, for example, those where customer goodwill is reduced by a disruption, the cost of the disruption may be hard to estimate. Therefore, controlling the amount of disruption is modeled as a constraint. However, there are other situations, for example, those where a disruption mainly increases internal production costs, where disruption costs are easier to estimate. Here the modeling of disruption costs as part of an objective, along with the usual scheduling costs, effectively evaluates the trade-off between scheduling and disruption costs.

General references on scheduling include Conway et al. (1967), Pinedo (1995), and Brucker (1998). Models for revising existing schedules to account for disruptions, but without considering the differences between the original and new schedules, are provided by Chang et al. (1984) and Donath and Graves (1985). Bean et al. (1991) develop a procedure that enables the new schedule to match up with the previous schedule in an efficient way over time. Wu et al. (1993) suggest a composite objective rescheduling approach for a single-machine problem. The costs they consider are an earliness cost, a delay cost, and a sequence-change cost. Li et al. (1993) develop an iterative two-step rescheduling procedure.

There are several papers on rescheduling approaches in flexible manufacturing systems. Raman et al. (1989) develop a branch-and-bound procedure to reschedule a flexible manufacturing system in the presence of dynamic job arrivals. Church and Uzsoy (1992) address a similar

problem for which they describe periodic rescheduling policies and analyze their error bounds. Jain and Elmaraghy (1997) use genetic algorithms to develop heuristic approaches for rescheduling a flexible manufacturing system.

There are also approaches to dealing with disruptions that emphasize the design of *robust* schedules that will not be sensitive to future disruptions. Daniels and Kouvelis (1995) develop robust schedules that protect against processing time uncertainty in single-stage production. Yellig and Mackulak (1997) develop a procedure based on historic machine performance that reduces the need for frequent rescheduling, or scheduling nervousness, which results from some other approaches. Mehta and Uzsoy (1999) describe a preventive scheduling approach that reduces the effect of disruptions on job completion times, while maintaining good scheduling performance, through the insertion of idle time. O'Donovan et al. (1999) develop robust preventive schedules that can absorb disruptions easily and also achieve good scheduling performance.

Within the rescheduling literature, two papers are closely related to our work. Wu et al. (1992) consider the problem of rescheduling a job shop after a disruption, to minimize the makespan and the disruption from the original schedule. Their measure of disruption is the difference between the start times of operations in the original and new schedules. Unal et al. (1997) consider a single machine with newly arrived jobs that have setup times that depend on their part types. They consider inserting new jobs into the original schedule so as to minimize the total weighted completion time or makespan of the new jobs without incurring additional setups or causing jobs to become late. They provide efficient algorithms, intractability results, and heuristics with guaranteed performance bounds. In our work, we consider measures of scheduling cost other than makespan, and also different measures of disruption. We also calculate our scheduling cost over both the original and the new jobs.

This paper is organized as follows. In §2, we introduce some basic definitions and notation, and provide an overview of our algorithm and computational complexity results. Section 3 provides structural results that are useful for several problems. Section 4 considers scheduling models with a constraint on the amount of disruption that can occur as a result of a single arrival of several new jobs. For the classical objective functions of minimizing maximum lateness and total completion time, and several practical measures of disruption, we provide either an efficient algorithm or a proof that such an algorithm is unlikely to exist. Similar results are derived in §5 for scheduling models that minimize the total of a classical scheduling cost and a cost of disruption. In §6, we show how some of the results of §§4 and 5 can be extended to deal with repeated arrivals of new jobs. Here, due to previous disruptions, the original schedule is no longer optimal with respect to the scheduling cost. Finally, §7 contains a conclusion and some suggestions for future research.

2. Preliminaries

2.1. Notation and Classification

Let $J_O = \{1, \dots, n_O\}$ denote a set of original jobs to be processed nonpreemptively on a single machine. In the models considered in §§4 and 5, we assume that these jobs have been scheduled optimally to minimize some classical objective and that π^* is an optimal schedule with no idle time between the jobs. Let $J_N = \{n_O + 1, \dots, n_O + n_N\}$ denote a set of new jobs that arrive together. We assume that these jobs arrive at time zero after a schedule for the jobs of J_O has been determined, but before processing begins. There is no loss of generality in this assumption: If the jobs arrive after time zero, then the fully processed jobs of J_O are removed from the problem, any partly processed jobs are processed to completion, and J_O and n_O are updated accordingly. Let $J = J_O \cup J_N$ and $n = n_O + n_N$. Let p_j denote the positive processing time of job j for $j = 1, \dots, n$. Some problems also have a due date d_j for each job j . We assume throughout that all p_j and d_j are known integers. We let $P_O = \sum_{j \in J_O} p_j$ and $P_N = \sum_{j \in J_N} p_j$.

Let $\nu \in \{\pi^*, \pi\}$, where π^* and π denote an optimal schedule and an arbitrary schedule of the jobs of J_O , respectively. For any schedule σ of the jobs of J we define the following variables:

$$\begin{aligned} C_j(\sigma) &= \text{the time at which job } j \text{ is completed for } j \in J, \\ L_j(\sigma) &= C_j(\sigma) - d_j, \text{ the lateness of job } j \text{ for } j \in J, \\ D_j(\nu, \sigma) &= \text{the sequence disruption of job } j \text{ for } j \in J_O, \\ \Delta_j(\nu, \sigma) &= |C_j(\sigma) - C_j(\nu)|, \text{ the time disruption of job } \\ &\quad j \text{ for } j \in J_O, \end{aligned}$$

where the sequence disruption of job j in schedule σ is the absolute value of the difference between the positions of that job in σ and ν . When there is no ambiguity, we simplify $C_j(\sigma)$, $L_j(\sigma)$, $D_j(\nu, \sigma)$, and $\Delta_j(\nu, \sigma)$ to C_j , L_j , $D_j(\nu)$, and $\Delta_j(\nu)$, respectively. Both types of disruptions model penalties associated with the change of delivery times of jobs to customers and the cost of rescheduling the resources so that they are available when required.

The standard classification scheme for scheduling problems (Graham et al. 1979) is $\alpha|\beta|\gamma$, where α indicates the scheduling environment, β describes the job characteristics or restrictive requirements, and γ defines the objective function to be minimized. We consider only single-machine problems, thus implying that $\alpha = 1$.

Under β , we indicate a constraint on the amount of disruption where applicable. Such constraints include the following, where $k \geq 0$ is a known integer and $\nu \in \{\pi^*, \pi\}$:

$D_{\max}(\nu) \leq k$: $\max_{j \in J_O} \{D_j(\nu)\} \leq k$, the maximum sequence disruption of the jobs cannot exceed k ,

$\sum D_j(\nu) \leq k$: $\sum_{j \in J_O} D_j(\nu) \leq k$, the total sequence disruption of the jobs cannot exceed k ,

$\Delta_{\max}(\nu) \leq k$: $\max_{j \in J_O} \{\Delta_j(\nu)\} \leq k$, the maximum time disruption of the jobs cannot exceed k ,

$\sum \Delta_j(\nu) \leq k$: $\sum_{j \in J_O} \Delta_j(\nu) \leq k$, the total time disruption of the jobs cannot exceed k .

If $\nu = \pi^*$ as in §§4 and 5, then we consider a single disruption, before which the original schedule is optimal with respect to the classical scheduling objective. If $\nu = \pi$ as in §6, then we consider the most recent of multiple disruptions, before which the schedule is regarded as arbitrary due to the previous disruptions.

Let $\mathcal{S}$ denote the scheduling cost, let $\mathcal{D}$ denote the amount of disruption, and where relevant, let μ denote the relative cost of one unit of disruption compared to the scheduling cost, where $\mu \geq 0$. In our models, $\mathcal{S} \in \{L_{\max}, \sum_{j \in J} C_j\}$, where $L_{\max} = \max_{j \in J} \{L_j\}$ is the maximum lateness of the jobs, and $\sum_{j \in J} C_j$ is the total completion time of the jobs. Also, $\mathcal{D} \in \{D_{\max}(\nu), \sum_{j \in J_O} D_j(\nu), \Delta_{\max}(\nu), \sum_{j \in J_O} \Delta_j(\nu)\}$, where $D_{\max}(\nu) = \max_{j \in J_O} \{D_j(\nu)\}$ and $\Delta_{\max}(\nu)$ is defined similarly. Thus, the objective functions that we consider under γ require the minimization of either $\mathcal{S}$ or $\mathcal{S} + \mu\mathcal{D}$. Let σ^* denote an optimal schedule.

We use the following scheme to name the algorithms that we describe in §§4 and 5. First, “L” denotes maximum lateness, or “C” denotes total completion time, as specified by the scheduling objective. Where the objective function is of the form $\mathcal{S} + \mu\mathcal{D}$, we append either “+MSD” or “+TSD” to denote maximum or total sequence disruption, or “+MTD” or “+TTD” to denote maximum or total time disruption, respectively. Where the disruption is a constraint, we append the relevant disruption measure but omit the “+.”

2.2. Overview of Algorithm and Complexity Results

Table 1 summarizes our algorithm and computational complexity results. It is easy to see that the recognition versions of all the problems considered here belong to the class *NP*. The first column indicates whether we are considering a constrained (C) or total cost (T) formulation. The second column shows which of the four disruption measures described above is being used. The third and fourth columns summarize our algorithm and computational complexity results for the $L_{\max}$ and $\sum C_j$ scheduling objectives, respectively. We use *BNPC* or *UNPC* to denote that the recognition version of a problem is binary or unary *NP*-complete, respectively. Related definitions can be found in Garey and Johnson (1979). Where a polynomial or pseudopolynomial time algorithm exists, we indicate its time

complexity. Included in the appropriate cell in Table 1 is a reference to where in this paper a proof of that result can be found.

3. General Structural Results

Here we prove several preliminary results that are useful for a variety of problems.

LEMMA 1. Consider problems $1 \mid D_{\max}(\pi^*) \leq k \mid \gamma$ and $1 \mid \Delta_{\max}(\pi^*) \leq k \mid \gamma$, where $\gamma \in \{L_{\max}, \sum C_j\}$, and where there is an optimal priority index sequencing rule for the jobs of J_O that gives the same ordering for the jobs of J_O in π^* and in σ^* . Then, both problems have an optimal schedule with no idle time between jobs, and

(a) a schedule for problem $1 \mid D_{\max}(\pi^*) \leq k \mid \gamma$ is feasible if and only if the number of jobs of J_N scheduled before the last job of J_O is less than or equal to k ;

(b) a schedule for problem $1 \mid \Delta_{\max}(\pi^*) \leq k \mid \gamma$ is feasible if and only if the total processing time of jobs of J_N scheduled before the last job of J_O is less than or equal to k .

PROOF. Because the ordering of jobs of J_O is the same in π^* and σ^* , removing any idle time in σ^* maintains feasibility and does not increase the value of $L_{\max}$ or $\sum C_j$. The priority index sequencing rule shows that every job of J_O follows the same number of other jobs of J_O in σ^* as it does in π^* . Therefore, the value of $D_{\max}(\pi^*)$ is determined by the number of jobs of J_N that precede the last job of J_O , and the value of $\Delta_{\max}(\pi^*)$ is determined by the total processing time of jobs of J_N that precede the last job of J_O . $\square$

We now discuss two classical scheduling rules that are useful in analyzing our models. There is no idle time between the jobs in either case.

EDD Rule. In the earliest due date (EDD) rule, jobs are sequenced in nondecreasing order of their due dates. Jackson (1955) shows that the EDD rule provides an optimal schedule for the classical problem $1 \parallel L_{\max}$. Thus, when the scheduling objective is to minimize the maximum lateness, we assume that the jobs of J_O are sequenced in EDD order in π^* .

SPT Rule. In the shortest processing time (SPT) rule, jobs are sequenced in nondecreasing order of their processing times. Smith (1956) shows that the SPT rule provides

Table 1. The complexity of rescheduling problems.

C/T	Measure	$L_{\max}$		$\sum C_j$	
C	$D_{\max}$	$O(n + n_N \log n_N)$	Theorem 1	$O(n + n_N \log n_N)$	Theorem 4
	$\sum D_j$	Open		$O(n_O^2 n_N^2)$	Theorem 5
	$\Delta_{\max}$	$O(n + n_N \log n_N)$	Theorem 2	$O(n + n_N \log n_N)$	Theorem 6
	$\sum \Delta_j$	UNPC	Theorem 3	$O(n_O n_N \min\{n_O P_N, n_N P_O\})$, BNPC	Theorems 7,8
T	$D_{\max}$	$O(n + n_N \log n_N)$	Theorem 9	$O(n + n_N \log n_N)$	Theorem 12
	$\sum D_j$	Open		$O(n_N \log n)$	Theorem 13
	$\Delta_{\max}$	$O(n + n_N \log n_N)$	Theorem 10	$O(n + n_N \log n_N)$	Theorem 14
	$\sum \Delta_j$	UNPC	Theorem 11	$O(n_N \log n)$	Theorem 15

an optimal schedule for the classical problem $1 \parallel \sum C_j$. Thus, when the scheduling objective is to minimize the total completion time, we assume that the jobs of J_O are sequenced in SPT order in π^* .

For notational convenience, we assume that the jobs are indexed so that schedule π^* is defined by the sequence $(1, \dots, n_O)$ with no idle time between jobs. Thus, $d_1 \leq \dots \leq d_{n_O}$ for the maximum lateness scheduling objective and $p_1 \leq \dots \leq p_{n_O}$ for the total completion time scheduling objective. We now show that the EDD and SPT rules apply to several of the rescheduling problems we consider.

LEMMA 2. *For problems $1 \mid D_{\max}(\pi^*) \leq k \mid L_{\max} + \mu D_{\max}(\pi^*)$ and $1 \mid \Delta_{\max}(\pi^*) \leq k \mid L_{\max} + \mu \Delta_{\max}(\pi^*)$, there exists an optimal schedule in which the jobs of J_O are sequenced in EDD order as in π^* , the jobs of J_N are sequenced in EDD order, and there is no idle time between jobs.*

PROOF. We first analyze the jobs of J_O . Consider an optimal schedule σ^* in which the jobs of J_O are not sequenced in EDD order as in π^* . Let i be the job with the smallest index that appears later relative to the other jobs of J_O in σ^* than in π^* , and let $j > i$ be the last job of J_O that precedes i in σ^* . Because π^* is an EDD schedule, $d_i \leq d_j$. Consider a new schedule σ' that is obtained from σ^* by removing job j from its original position and inserting it immediately after job i , so that job i and all jobs between j and i in σ^* are completed p_j units earlier in σ' than in σ^* . Because the lateness of job j in σ' is no greater than the lateness of job i in σ^* , the maximum lateness of the jobs does not increase as a result of moving job j . If the position of job j in σ' is greater than or equal to its position in π^* (as is the case for job i), then $D_i(\pi^*, \sigma') = D_i(\pi^*, \sigma^*) - 1$ and $D_j(\pi^*, \sigma') = D_j(\pi^*, \sigma^*) - (j - i)$, and therefore $D_{\max}(\pi^*, \sigma') \leq D_{\max}(\pi^*, \sigma^*)$. Alternatively, if the position of job j in σ' is less than its position in π^* , then $D_i(\pi^*, \sigma') = D_i(\pi^*, \sigma^*) - 1$ and $D_j(\pi^*, \sigma') < D_j(\pi^*, \sigma^*)$, and hence $D_{\max}(\pi^*, \sigma') \leq D_{\max}(\pi^*, \sigma^*)$. If $C_j(\sigma') \geq C_j(\pi^*)$ (note that $C_i(\sigma') \geq C_i(\pi^*)$), then $\Delta_i(\pi^*, \sigma') < \Delta_i(\pi^*, \sigma^*)$ and $\Delta_j(\pi^*, \sigma') < \Delta_j(\pi^*, \sigma^*)$, and therefore $\Delta_{\max}(\pi^*, \sigma') \leq \Delta_{\max}(\pi^*, \sigma^*)$. Alternatively, if $C_j(\sigma') < C_j(\pi^*)$, then $\Delta_i(\pi^*, \sigma') < \Delta_i(\pi^*, \sigma^*)$ and $\Delta_j(\pi^*, \sigma') < \Delta_j(\pi^*, \sigma^*)$, and hence $\Delta_{\max}(\pi^*, \sigma') \leq \Delta_{\max}(\pi^*, \sigma^*)$. Thus, for either problem, σ' is feasible and optimal. A finite number of repetitions of this argument shows that there exists an optimal schedule in which the jobs of J_O are sequenced in EDD order as in π^* . A similar job-insertion argument establishes that the jobs of J_N can also be sequenced in EDD order. Because the jobs of J_O appear in the same EDD order in π^* and an optimal schedule, removing any idle time in this optimal schedule maintains feasibility and does not increase the maximum lateness. $\square$

LEMMA 3. *For problems $1 \mid D_{\max}(\pi^*) \leq k \mid \sum C_j + \mu D_{\max}(\pi^*)$, $1 \mid \sum D_j(\pi^*) \leq k \mid \sum C_j + \mu \sum D_j(\pi^*)$, $1 \mid \Delta_{\max}(\pi^*) \leq k \mid \sum C_j + \mu \Delta_{\max}(\pi^*)$, and $1 \mid \sum \Delta_j(\pi^*) \leq k \mid \sum C_j +$*

$\mu \sum \Delta_j(\pi^)$, there exists an optimal schedule in which the jobs of J_O are sequenced in SPT order as in π^* , the jobs of J_N are sequenced in SPT order, and there is no idle time between jobs.*

PROOF. We first analyze the jobs of J_O . Consider an optimal schedule σ^* in which the jobs of J_O are not sequenced in SPT order as in π^* . Let i be the job with the smallest index that appears later relative to the other jobs of J_O in σ^* than in π^* , and let $j > i$ be the last job of J_O that precedes job i in σ^* . Because π^* is an SPT schedule, $p_i \leq p_j$. Consider a new schedule σ' that is obtained from σ^* by interchanging jobs i and j , so that $C_i(\sigma') = C_j(\sigma^*) - p_j + p_i \leq C_j(\sigma^*)$, $C_j(\sigma') = C_i(\sigma^*)$, and all jobs between j and i are completed $p_j - p_i \geq 0$ units earlier in σ' than in σ^* . A straightforward computation shows that the total completion time does not increase as a result of the interchange. If the position of job j in σ' is greater than or equal to its position in π^* (as is the case for job i), then $D_j(\pi^*, \sigma') < D_j(\pi^*, \sigma^*)$; otherwise, $D_j(\pi^*, \sigma') < D_j(\pi^*, \sigma^*)$. In either case, because $D_i(\pi^*, \sigma') = D_i(\pi^*, \sigma^*) - h$ and $D_j(\pi^*, \sigma') \leq D_j(\pi^*, \sigma^*) + h$, where h is the difference between the positions of jobs i and j in σ^* , we deduce that $D_{\max}(\pi^*, \sigma') \leq D_{\max}(\pi^*, \sigma^*)$ and $\sum D_j(\pi^*, \sigma') \leq \sum D_j(\pi^*, \sigma^*)$. Moreover, if $C_j(\sigma') \geq C_j(\pi^*)$, then $\Delta_j(\pi^*, \sigma') < \Delta_j(\pi^*, \sigma^*)$; otherwise, $\Delta_j(\pi^*, \sigma') < \Delta_j(\pi^*, \sigma^*)$. In either case, because $\Delta_i(\pi^*, \sigma') = \Delta_i(\pi^*, \sigma^*) - h'$ and $\Delta_j(\pi^*, \sigma') \leq \Delta_j(\pi^*, \sigma^*) + h'$, where $h' = C_i(\sigma^*) - C_i(\sigma')$, we deduce that $\Delta_{\max}(\pi^*, \sigma') \leq \Delta_{\max}(\pi^*, \sigma^*)$ and $\sum \Delta_j(\pi^*, \sigma') \leq \sum \Delta_j(\pi^*, \sigma^*)$. Thus, σ' is feasible and optimal. A finite number of repetitions of this argument shows that there exists an optimal schedule in which the jobs of J_O are sequenced in SPT order as in π^* . A similar interchange argument establishes that the jobs of J_N can also be sequenced in SPT order. The same SPT ordering of the jobs of J_O in π^* and an optimal schedule shows that there is no idle time in this optimal schedule: Otherwise, removing this idle time maintains feasibility and decreases the total completion time. $\square$

Lemmas 2 and 3 are useful in developing algorithms for several of the problems in this paper. We refer to the (EDD, EDD) property when a schedule is constructed using Lemma 2 and to the (SPT, SPT) property when a schedule is constructed using Lemma 3. Also, a *no-idle schedule* has no machine idle time and is therefore defined by a sequence of the jobs. All of the algorithms derived in §§4 and 5 are for special cases of the problems specified in Lemmas 2 and 3. Consequently, an optimal schedule is no-idle, and therefore it is sufficient to specify a sequence of jobs.

4. Single-Disruption Constrained Problems

In this section, we analyze problems of minimizing a classical scheduling cost objective, subject to a limit on the disruption to the original schedule caused by the arrival of a

single set of new jobs. The no-idle schedule of the original jobs is denoted by π^* . Gershwin (1987) presents a hierarchical framework for scheduling discrete events, which in our model represent the arrival of a set of new jobs. His assumption that when an event occurs it prevents all other events from occurring for a period of time motivates our study of single-disruption models.

We first consider problem $1 \mid D_{\max}(\pi^*) \leq k \mid L_{\max}$. Applying the (EDD, EDD) property of Lemma 2 within Lemma 1 shows that up to k jobs of J_N can be sequenced before the last job of J_O , and that these jobs have the smallest due dates. Thus, we propose the following algorithm.

Algorithm LMSD

Input

Given p_j, d_j for $j = 1, \dots, n, k$ and π^* , where $k \leq n_N$.

Indexing

Index the jobs of J_N in EDD order.

Schedule Construction

Schedule jobs $1, \dots, n_O + k$ in EDD order in the first $n_O + k$ positions.

Schedule jobs $n_O + k + 1, \dots, n$ in EDD order in the final $n_N - k$ positions.

THEOREM 1. *Algorithm LMSD finds an optimal schedule for problem $1 \mid D_{\max}(\pi^*) \leq k \mid L_{\max}$ in $O(n + n_N \log n_N)$ time.*

PROOF. From Lemmas 1 and 2, the constraint $D_{\max}(\pi^*) \leq k$ allows up to k jobs of J_N to be sequenced before the final job of J_O , and these are the jobs of J_N with the smallest due dates. Classical scheduling theory (Jackson 1955) shows that the jobs of this first group are sequenced in EDD order, while Lemma 2 establishes that the remaining $n_N - k$ jobs of J_N are also sequenced in EDD order.

To establish the time complexity of Algorithm LMSD, we note that the Indexing step for the jobs of J_N requires $O(n_N \log n_N)$ time. The Schedule Construction step is executed in $O(n)$ time by merging the first k jobs of the EDD-ordered jobs of J_N with the jobs of J_O as sequenced in π^* and then placing the last $n_N - k$ jobs of the EDD-ordered jobs of J_N at the end of the schedule. $\square$

The computational complexity of problem $1 \mid \sum D_j(\pi^*) \leq k \mid L_{\max}$ remains open.

We next consider problem $1 \mid \Delta_{\max}(\pi^*) \leq k \mid L_{\max}$. Because the (EDD, EDD) property of Lemma 2 again holds, $C_j \geq C_j(\pi^*)$ for $j \in J_O$. Thus, the constraint reduces to $C_j \leq C_j(\pi^*) + k$, or, equivalently, job j has a deadline $\bar{d}_j = C_j(\pi^*) + k$ for $j \in J_O$. The resulting problem is one of minimizing the maximum lateness, subject to deadline constraints, i.e., problem $1 \mid \bar{d}_j \mid L_{\max}$, where $\bar{d}_j = \infty$ for $j \in J_N$. Koulamas and Kyparisis (2001) show that an optimal schedule for this problem can be found by a backward scheduling procedure in which at each iteration a feasible

job with the largest due date is assigned to the last empty position in the sequence.

THEOREM 2. *An optimal schedule for problem $1 \mid \Delta_{\max}(\pi^*) \leq k \mid L_{\max}$ can be found in $O(n + n_N \log n_N)$ time.*

PROOF. For the constructed instance of problem $1 \mid \bar{d}_j \mid L_{\max}$, the deadlines and due dates for the jobs of J_O are consistent; i.e., $d_1 \leq \dots \leq d_{n_O}$ and $\bar{d}_1 \leq \dots \leq \bar{d}_{n_O}$. Thus, the algorithm of Koulamas and Kyparisis (2001) schedules the jobs according to the (EDD, EDD) property.

To implement the algorithm of Koulamas and Kyparisis (2001) efficiently, the jobs of J_N are first listed in EDD order, which requires $O(n_N \log n_N)$ time. The job to be scheduled in the last empty position in the sequence is either the last unscheduled job in π^* or the last unscheduled job in the EDD-listed jobs of J_N , depending on the deadlines and the due dates of these two jobs. Thus, having constructed the list of jobs of J_N in $O(n_N \log n_N)$ time, the schedule is constructed in $O(n)$ time, and the stated time complexity follows. $\square$

We next consider problem $1 \mid \sum \Delta_j(\pi^*) \leq k \mid L_{\max}$.

THEOREM 3. *The recognition version of problem*

$$1 \mid \sum \Delta_j(\pi^*) \leq k \mid L_{\max}$$

is unary NP-complete.

PROOF. The proof is by reduction from the following problem, which is known to be unary NP-complete.

3-PARTITION PROBLEM (GAREY AND JOHNSON 1979). Given $3t$ elements with integer sizes $a_1, \dots, a_{3t}$, where $\sum_{i=1}^{3t} a_i = ty$ and $y/4 < a_i < y/2$ for $i = 1, \dots, 3t$, does there exist a partition $S_1, \dots, S_t$ of the index set $\{1, \dots, 3t\}$ such that $|S_j| = 3$ and $\sum_{i \in S_j} a_i = y$ for $j = 1, \dots, t$?

We assume without loss of generality that if there exists a solution to the 3-Partition Problem, then the elements are numbered such that $a_{3i-2} + a_{3i-1} + a_{3i} = y$ for $i = 1, \dots, t$.

Consider the following instance of the recognition version of problem $1 \mid \sum \Delta_j(\pi^*) \leq k \mid L_{\max}$:

$$n_O = 2t$$

$$n_N = 3t$$

$$p_j = 1, \quad j = 1, \dots, t$$

$$d_j = t(2ty + 1), \quad j = 1, \dots, t$$

$$p_j = ty, \quad j = t + 1, \dots, 2t$$

$$d_j = t(2ty + 1), \quad j = t + 1, \dots, 2t$$

$$p_j = ta_{j-2t}, \quad j = 2t + 1, \dots, 5t$$

$$d_j = t^2y + t - 1, \quad j = 2t + 1, \dots, 5t$$

$$k = t^3y + t(t + 1)/2$$

$$C = 0.$$

We prove that there exists a feasible schedule for this instance of problem $1 \mid \sum \Delta_j(\pi^*) \leq k \mid L_{\max}$ with $L_{\max} \leq C$

if and only if there exists a solution to the 3-Partition Problem. Because all due dates for the jobs of J_O are identical, we assume that the no-idle schedule π^* is defined by the sequence $(t+1, 1, t+2, 2, \dots, 2t, t)$.

($\Rightarrow$) Consider the no-idle schedule σ defined by the sequence $(2t+1, 2t+2, 2t+3, 1, 2t+4, 2t+5, 2t+6, 2, \dots, 5t-2, 5t-1, 5t, t, t+1, t+2, \dots, 2t)$. Note that $C_j(\sigma) = C_j(\pi^*)$ for $j = 1, \dots, t$. Thus,

$$\begin{aligned} \sum_{j \in J_O} \Delta_j(\pi^*, \sigma) &= [t(ty+1) + ty - ty] \\ &\quad + [t(ty+1) + 2ty - (2ty+1)] \\ &\quad + \dots + [t(ty+1) + t^2y - (t^2y+t-1)] = k. \end{aligned}$$

Moreover, $L_{\max}(\sigma) = 0 = C$.

($\Leftarrow$) Consider a feasible schedule σ with $L_{\max}(\sigma) \leq 0$. Because $\sum_{j \in J_N} p_j = t^2y = \max_{j \in J_N} \{d_j\} - (t-1)$, no job j where $j \in \{t+1, \dots, 2t\}$, and therefore $p_j \geq t$, can be sequenced before any job of J_N in σ .

Suppose that exactly h jobs of the set $\{1, \dots, t\}$ are scheduled after the first job of $\{t+1, \dots, 2t\}$ in σ . Each such job completes in σ no earlier than time $t^2y + ty + (t-h) + 1$, whereas it completes in π^* no later than time $t(ty+1)$. Moreover, jobs $t+1, \dots, 2t$ are completed at times $ty, 2ty+1, \dots, t^2y+t-1$ in π^* , and are preceded by all jobs of J_N and $t-h$ jobs of $\{1, \dots, t\}$ and therefore are completed no earlier than times $t^2y + (t-h) + ty, t^2y + (t-h) + 2ty, \dots, t^2y + (t-h) + t^2y$ in σ . Therefore,

$$\begin{aligned} \sum_{j \in J_O} \Delta_j(\pi^*, \sigma) &\geq [t^2y + (t-h) + ty - ty] \\ &\quad + [t^2y + (t-h) + 2ty - (2ty+1)] \\ &\quad + \dots + [t^2y + (t-h) + t^2y - (t^2y+t-1)] \\ &\quad + h[t^2y + ty + (t-h) + 1 - (t^2y+t)] \\ &= t^3y + t(t-h) - t(t-1)/2 + h(ty-h+1) \\ &= k + h(ty-h-t+1). \end{aligned}$$

Because $y \geq 3$ from the definition of the 3-Partition Problem, and schedule σ is feasible, we deduce that $h = 0$. Moreover, $\sum_{j=t+1}^{2t} \Delta_j(\pi^*, \sigma) = k$, which implies that $\Delta_j(\pi^*, \sigma) = 0$ for $j = 1, \dots, t$. Thus, jobs $1, \dots, t$ are processed in the same intervals $[ty, ty+1], [2ty+1, 2ty+2], \dots, [t^2y+t-1, t^2y+t]$ as in π^* . Because $L_{\max}(\sigma) \leq 0$, the jobs of J_N are scheduled in the intervals $[0, ty], [ty+1, 2ty+1], \dots, [(t-1)ty+t-1, t^2y+t-1]$. This implies the existence of a solution to the 3-Partition Problem. $\square$

Turning to problems with $\sum C_j$ objective, we first consider problem $1 \mid D_{\max}(\pi^*) \leq k \mid \sum C_j$. In view of Lemmas 1 and 3, we propose Algorithm CMSD, which is identical to Algorithm LMSD except that the EDD order is replaced by the SPT order throughout.

THEOREM 4. *Algorithm CMSD finds an optimal schedule for problem $1 \mid D_{\max}(\pi^*) \leq k \mid \sum C_j$ in $O(n + n_N \log n_N)$ time.*

PROOF. The proof is similar to that of Theorem 1. $\square$

We next consider problem $1 \mid \sum D_j(\pi^*) \leq k \mid \sum C_j$. The (SPT, SPT) property of Lemma 3 shows that an optimal schedule is found by merging the SPT-ordered lists of jobs of J_O and of J_N . The following dynamic programming algorithm performs an optimal merging, subject to the given constraint on the total sequence disruption.

Algorithm CTSD

Input

Given p_j , for $j = 1, \dots, n$, k and π^* , where $k \leq n_O n_N$.

Indexing

Index the jobs of J_N in SPT order.

Preprocessing

Compute $\sum_{h=1}^i p_h$ for $i = 1, \dots, n_O$ and $\sum_{h=n_O+1}^{n_O+j} p_h$ for $j = 1, \dots, n_N$.

Value Function

$f(i, j, \delta)$ = minimum total completion time of a partial schedule for jobs $1, \dots, i$ and $n_O + 1, \dots, n_O + j$, where the total sequence disruption is equal to δ .

Boundary Condition

$f(0, 0, 0) = 0$.

Optimal Solution Value

$\min_{0 \leq \delta \leq k} \{f(n_O, n_N, \delta)\}$

Recurrence Relation

$f(i, j, \delta) = \sum_{h=1}^i p_h + \sum_{h=n_O+1}^{n_O+j} p_h + \min\{f(i-1, j, \delta-j), f(i, j-1, \delta)\}$.

In the recurrence relation, the first term in the minimization corresponds to the case where the partial schedule ends with job $i \in J_O$. Because j jobs of J_N appear before job i in such a partial schedule, the increase in total sequence disruption is equal to j . The second term corresponds to the case where the partial schedule ends with job $n_O + j \in J_N$.

THEOREM 5. *Algorithm CTSD finds an optimal schedule for problem $1 \mid \sum D_j(\pi^*) \leq k \mid \sum C_j$ in $O(n_O^2 n_N^2)$ time.*

PROOF. From Lemma 3, it only remains to enumerate all possible ways of merging the SPT-ordered lists of jobs of J_O and J_N . Algorithm CTSD does so by comparing the cost of all possible state transitions, and therefore finds an optimal schedule.

We now consider the time complexity of Algorithm CTSD. Because $i \leq n_O$, $j \leq n_N$, and $\delta \leq k \leq n_O n_N$, there are $O(n_O^2 n_N^2)$ values of the state variables. Indexing the jobs of J_N in SPT order, and evaluating the partial sums $\sum_{h=1}^i p_h$ for $i = 1, \dots, n_O$ and $\sum_{h=n_O+1}^{n_O+j} p_h$ for $j = 1, \dots, n_N$ in the Indexing and Preprocessing steps requires $O(n + n_N \log n_N)$ time. The recurrence relation requires constant time for each set of values of the state variables. Thus, the overall time complexity of Algorithm CTSD is $O(n_O^2 n_N^2)$. $\square$

We next consider problem $1 \mid \Delta_{\max}(\pi^*) \leq k \mid \sum C_j$, for which our analysis is similar to that for problem $1 \mid \Delta_{\max}(\pi^*) \leq k \mid L_{\max}$. The (SPT, SPT) property of Lemma 3 again holds, so that $C_j \geq C_j(\pi^*)$ for $j \in J_O$, and therefore the constraint reduces to $C_j \leq C_j(\pi^*) + k$ for $j \in J_O$. The resulting problem is one of minimizing total completion time, subject to deadline constraints, i.e., problem $1 \mid \bar{d}_j \mid \sum C_j$, where $\bar{d}_j = C_j(\pi^*) + k$ for $j \in J_O$ and $\bar{d}_j = \infty$ for $j \in J_N$. Smith (1956) shows that this problem is solved by a backward scheduling procedure in which at each iteration a feasible job with largest processing time is sequenced last.

THEOREM 6. *An optimal schedule for problem $1 \mid \Delta_{\max}(\pi^*) \leq k \mid \sum C_j$ can be found in $O(n + n_N \log n_N)$ time.*

PROOF. For the constructed instance of problem $1 \mid \bar{d}_j \mid \sum C_j$, the processing times and deadlines of the jobs of J_O are consistent; i.e., $p_1 \leq \dots \leq p_{n_O}$ and $\bar{d}_1 \leq \dots \leq \bar{d}_{n_O}$. Thus, the algorithm of Smith (1956) schedules the jobs according to the (SPT, SPT) property.

We use a similar implementation to that described in the proof of Theorem 2 for problem $1 \mid \Delta_{\max}(\pi^*) \leq k \mid L_{\max}$. The jobs of J_N are indexed in SPT order, so that the job to be scheduled in the last unfilled position in the sequence is either the last unscheduled job in π^* or the last unscheduled job among the SPT-indexed jobs of J_N . Because the ordering of the jobs of J_N requires $O(n_N \log n_N)$ time and the schedule construction requires $O(n)$ time, the overall time complexity of $O(n + n_N \log n_N)$ follows. $\square$

Finally, we consider problem $1 \mid \sum \Delta_j(\pi^*) \leq k \mid \sum C_j$, for which we again use the (SPT, SPT) property of Lemma 3. The following dynamic programming algorithm performs an optimal merging of the jobs of J_O and J_N in a way similar to that in Algorithm CTSD.

Algorithm CTTD

Input

Given p_j , for $j = 1, \dots, n$, k and π^* , where $k \leq n_O P_N$.

Indexing

Index the jobs of J_N in SPT order.

Preprocessing

Compute $\sum_{h=1}^i p_h$ for $i = 1, \dots, n_O$ and $\sum_{h=n_O+1}^{n_O+j} p_h$ for $j = 1, \dots, n_N$.

Value Function

$f(i, j, \tau)$ = minimum total completion time of a partial schedule for jobs $1, \dots, i$ and $n_O + 1, \dots, n_O + j$, where the total time disruption is equal to τ .

Boundary Condition

$f(0, 0, 0) = 0$.

Optimal Solution Value

$\min_{0 \leq \tau \leq k} \{f(n_O, n_N, \tau)\}$.

Recurrence Relation

$$f(i, j, \tau) = \sum_{h=1}^i p_h + \sum_{h=n_O+1}^{n_O+j} p_h + \min\{f(i-1, j, \tau - \sum_{h=n_O+1}^{n_O+j} p_h), f(i, j-1, \tau)\}.$$

In the recurrence relation, the first term in the minimization corresponds to the case where the partial schedule ends with job $i \in J_O$. Because j jobs of J_N with total processing time $\sum_{h=n_O+1}^{n_O+j} p_h$ have already been scheduled, that quantity is the increase in the total time disruption resulting from this decision. The second term corresponds to the case where the partial schedule ends with job $n_O + j \in J_N$.

THEOREM 7. *An optimal schedule for problem $1 \mid \sum \Delta_j(\pi^*) \leq k \mid \sum C_j$ can be found in $O(n_O n_N \min\{n_O P_N, n_N P_O\})$ time.*

PROOF. The proof of optimality of Algorithm CTTD is similar to that in Theorem 5. Regarding the time complexity, because $i \leq n_O$, $j \leq n_N$, and $\tau \leq k \leq n_O P_N$, there are $O(n_O^2 n_N P_N)$ values of the state variables. Similar arguments to those in the proof of Theorem 5 show that the overall time complexity of Algorithm CTTD is $O(n_O^2 n_N P_N)$.

By reversing the roles of J_O and J_N , we can regard the jobs of J_N as having a time disruption. We can show that for any no-idle schedule where the (SPT, SPT) property holds, the total completion time is equal to the total time disruption of the jobs of J_O , plus the total time disruption of the jobs of J_N , plus a constant. Thus, we can define an alternative algorithm that is similar to Algorithm CTTD where i, j , and the total time disruption of the jobs of J_N are state variables, and the constraint on the total time deviation of the jobs of J_O is implemented in the Optimal Solution Value step. The time complexity of this alternative algorithm is $O(n_O n_N^2 P_O)$. $\square$

We now show that a pseudopolynomial time algorithm, as established in Theorem 7, is the best possible type of result, unless $P = NP$.

THEOREM 8. *The recognition version of problem $1 \mid \sum \Delta_j(\pi^*) \leq k \mid \sum C_j$ is binary NP-complete.*

PROOF. The proof is by reduction from the following problem, which is known to be binary NP-complete.

EVEN-ODD PARTITION PROBLEM (GAREY ET AL. 1988). Given a set $\{a_1, \dots, a_{2t}\}$ of positive integers, where $a_i \leq a_{i+1}$ for $1 \leq i \leq 2t-1$ and $\sum_{i=1}^{2t} a_i = 2A$, does there exist a partition of the index set $S = \{1, \dots, 2t\}$ into subsets S_1 and S_2 , such that $\sum_{i \in S_1} a_i = \sum_{i \in S_2} a_i = A$, and that S_1 and S_2 each contains exactly one of the indices $2i-1$ and $2i$ for $i = 1, \dots, t$?

If there exists a solution to a given instance of the Even-Odd Partition Problem, then we let $\delta_1, \dots, \delta_t$ denote such a partition, where $\delta_i = 0$ if $2i-1 \in S_1$ and $\delta_i = 1$ if $2i \in S_1$ for $i = 1, \dots, t$; i.e., $\sum_{i=1}^t a_{2i-1}(1 - \delta_i) + \sum_{i=1}^t a_{2i}\delta_i = \sum_{i=1}^t a_{2i-1}\delta_i + \sum_{i=1}^t a_{2i}(1 - \delta_i)$.

Consider the following instance of the recognition version of problem 1 | $\sum \Delta_j(\pi^*) \leq k$ | $\sum C_j$:

$$n_O = 2t$$

$$n_N = 2t$$

$$p_j = 2B^{\lceil j/2 \rceil} + 2a_j, \quad j = 1, \dots, 2t$$

$$p_j = B^{\lceil (j-2t)/2 \rceil} + a_{j-2t}, \quad j = 2t+1, \dots, 4t$$

$$k = \sum_{j=1}^t (4t-4j+2)B^j + \sum_{j=1}^{2t} (2t-j)a_j + A$$

$$C = \sum_{j=1}^t (24(t-j) + 15)B^j + \sum_{j=1}^{2t} (12t-6j)a_j + 9A,$$

where $B = tA^3$.

We prove that there exists a feasible schedule for this instance of problem 1 | $\sum \Delta_j(\pi^*) \leq k$ | $\sum C_j$ with $\sum_{j \in J} C_j \leq C$ if and only if there exists a solution to the Even-Odd Partition Problem. Note that π^* is a no-idle schedule defined by the sequence $(1, \dots, 2t)$.

($\Rightarrow$) Consider the schedule σ defined by the sequence $(\{1, 2t+1\}, \{2, 2t+2\}, \dots, \{2t-1, 4t-1\}, \{2t, 4t\})$, where the order of jobs in each pair is now discussed. For $\delta_i = 0$, let the jobs in the $(2i-1)$ th and $2i$ th pairs be sequenced $(2i-1, 2t+2i-1, 2t+2i, 2i)$; for $\delta_i = 1$, let those jobs be sequenced $(2t+2i-1, 2i-1, 2i, 2t+2i)$.

If $\delta_1 = 0$, then the total time deviation caused by jobs $2t+1$ and $2t+2$ is $(2t-1)(B+a_1) + (2t-1)(B+a_2) = (4t-2)B + (2t-1/2)a_1 + (2t-3/2)a_2 + (a_2-a_1)/2$, because all jobs of J_O except job 1 are delayed. Alternatively, if $\delta_1 = 1$, then the total time deviation is $2t(B+a_1) + (2t-2)(B+a_2) = (4t-2)B + (2t-1/2)a_1 + (2t-3/2)a_2 + (a_1-a_2)/2$. Similarly, we obtain $\sum_{j \in J_O} \Delta_j(\pi^*, \sigma) = \sum_{j=1}^t (4t-4j+2)B^j + \sum_{j=1}^{2t} (2t-j+1/2)a_j - \sum_{i \in S_1} a_i/2 + \sum_{i \in S_2} a_i/2 = k$.

Further, if $\delta_1 = 0$, then the total completion time of jobs $\{1, 2, 2t+1, 2t+2\}$ is $(2B+2a_1) + (3B+3a_1) + (4B+3a_1+a_2) + (6B+3a_1+3a_2) = 15B+11a_1+4a_2 = 15B+21a_1/2+9a_2/2 + (a_1-a_2)/2$. Alternatively, if $\delta_1 = 1$, then the total completion time of the first four jobs is $(B+a_1) + (3B+3a_1) + (5B+3a_1+2a_2) + (6B+3a_1+3a_2) = 15B+10a_1+5a_2 = 15B+21a_1/2+9a_2/2 + (a_2-a_1)/2$. Moreover, these four jobs delay the start time of the remaining $4t-4$ jobs by $6B+3a_1+3a_2$. Similarly, we obtain $\sum_{j \in J} C_j(\sigma) = \sum_{j=1}^t (24(t-j) + 15)B^j + \sum_{j=1}^{2t} (12t-6j+9/2)a_j + \sum_{i \in S_1} a_i/2 - \sum_{i \in S_2} a_i/2 = C$.

($\Leftarrow$) Consider a schedule σ with $\sum_{j \in J_O} \Delta_j(\pi^*, \sigma) \leq k$ and $\sum_{j \in J} C_j(\sigma) \leq C$. Using Lemma 3, we assume that σ is a no-idle schedule with the jobs of J_O sequenced in the order $(1, \dots, 2t)$ and the jobs of J_N sequenced in the order $(2t+1, \dots, 4t)$.

We first show that in σ the jobs $4t-1$ and $4t$ are sequenced either in positions $4t-3$ and $4t$, or in positions $4t-2$ and $4t-1$, or in positions $4t-2$ and $4t$, or in positions $4t-1$ and $4t$, respectively. If these jobs appear in any other positions, then $\sum \Delta_j(\pi^*, \sigma) > 3B' > k$, which contradicts the feasibility of σ . We next eliminate two of the remaining four cases. First, if jobs $4t-1$ and $4t$ occupy

positions $4t-2$ and $4t$, and consequently job $2t$ is in position $4t-1$, then $\sum C_j(\sigma) > 2B' + 3B' + 5B' + 6B' = 16B' > C$. Second, if jobs $4t-1$ and $4t$ occupy positions $4t-1$ and $4t$, and consequently are preceded by jobs $2t-1$ and $2t$, then $\sum C_j(\sigma) > 2B' + 4B' + 5B' + 6B' = 17B' > C$. Thus, jobs $4t-1$ and $4t$ are sequenced in positions $4t-3$ and $4t$ with jobs $2t-1$ and $2t$ in between them, or in positions $4t-2$ and $4t-1$ with job $2t$ in position $4t$. In the latter case, job $2t-1$ occupies position $4t-3$, because otherwise job $2t-1$ precedes job $4t-2$, and consequently $\sum C_j(\sigma) > 2B' + 2B' + 3B' + 4B' + 6B' = 17B' > C$. We have thus established that $\sigma = (\dots, 4t-1, 2t-1, 2t, 4t)$ or $\sigma = (\dots, 2t-1, 4t-1, 4t, 2t)$.

Similar analysis for other groups of jobs $2i-1, 2i, 2t+2i-1$, and $2t+2i$ for $i = t-1, t-2, \dots, 1$, shows that they are sequenced in the order $(2i-1, 2t+2i-1, 2t+2i, 2i)$ or $(2t+2i-1, 2i-1, 2i, 2t+2i)$ in positions $4i-3, 4i-2, 4i-1, 4i$. We define sets S_1 and S_2 as follows. If the order $(2i-1, 2t+2i-1, 2t+2i, 2i)$ is contained in σ for $i = 1, \dots, t$, then $(2i-1) \in S_1$ and $2i \in S_2$; if the order $(2t+2i-1, 2i-1, 2i, 2t+2i)$ is contained in σ , then $2i \in S_1$ and $(2i-1) \in S_2$.

A computation that is similar to that in the ($\Rightarrow$) part of the proof shows that $\sum_{j \in J_O} \Delta_j(\pi^*, \sigma) = \sum_{j=1}^t (4t-4j+2)B^j + \sum_{j=1}^{2t} (2t-j+1/2)a_j - \sum_{i \in S_1} a_i/2 + \sum_{i \in S_2} a_i/2 = k - \sum_{i \in S_1} a_i/2 + \sum_{i \in S_2} a_i/2$. For feasibility, $\sum_{j \in J_O} \Delta_j(\pi^*, \sigma) \leq k$, and therefore $\sum_{i \in S_1} a_i/2 \geq \sum_{i \in S_2} a_i/2$.

Again using a similar computation, we obtain $\sum_{j \in J} C_j(\sigma) = \sum_{j=1}^t (24(t-j) + 15)B^j + \sum_{j=1}^{2t} (12t-6j+9/2)a_j + \sum_{i \in S_1} a_i/2 - \sum_{i \in S_2} a_i/2 = C + \sum_{i \in S_1} a_i/2 - \sum_{i \in S_2} a_i/2$. Because $\sum_{j \in J} C_j(\sigma) \leq C$, it follows that $\sum_{i \in S_1} a_i/2 \leq \sum_{i \in S_2} a_i/2$.

Thus, $\sum_{i \in S_1} a_i = \sum_{i \in S_2} a_i$, and the sets S_1 and S_2 define a solution to the Even-Odd Partition Problem. $\square$

5. Single-Disruption Total Cost Problems

In this section, we consider models for minimizing the total of a classical scheduling cost objective over all jobs and a cost of disruption caused by a single group of new jobs.

We first consider problem 1 | $L_{\max} + \mu D_{\max}(\pi^*)$. Our approach is to solve problem 1 | $D_{\max}(\pi^*) \leq k$ | $L_{\max}$ for $k = 0, \dots, n_N$ using Algorithm LMSD of §4. If σ_k denotes an optimal (no-idle) schedule for this constrained problem, then we compute $\min_{0 \leq k \leq n_N} \{L_{\max}(\sigma_k) + \mu k\}$. Our algorithm uses an efficient method to compute $L_{\max}(\sigma_k)$ over multiple values of k .

Algorithm L + MSD

Input

Given p_j, d_j for $j = 1, \dots, n$, μ and π^* .

Indexing

Index the jobs of J_N in EDD order and sequence all n jobs in EDD order.

Preprocessing

If job n_O is sequenced before job $n_O + 1$, then terminate.
Define j such that $n_O + j$ is the index of the last job of J_N that is sequenced before job n_O .

Compute the maximum lateness l_i of all jobs of J_O sequenced between jobs $n_O + i$ and $n_O + i + 1$ for $i = 1, \dots, j - 1$, and compute the maximum lateness l_j of all jobs of J_O sequenced after $n_O + j$ up to and including job n_O .

Compute the maximum lateness $\bar{l}_i$ of all jobs sequenced before $n_O + i$ for $i = 1, \dots, j$.

Compute the maximum lateness $\bar{l}$ of jobs $n_O + j + 1, \dots, n_O + n_N$ of J_N .

Set $L = -\infty$.

Set σ_j to be the schedule defined by the current job sequence, and compute $L_{\max}(\sigma_j)$.

Set t to be the completion time of job n_O .

Schedule Construction and Evaluation

Execute the following for $k = j, \dots, 1$.

Form schedule σ_{k-1} from σ_k by moving job $n_O + k$ immediately after job n_O .

Set $\tilde{L} = \max\{\bar{l}, t - d_{n_O+k}\}$ and $L = \max\{L, l_k\} - p_{n_O+k}$.

Compute $L_{\max}(\sigma_{k-1}) = \max\{\bar{l}_k, L, \bar{l}\}$.

Set $t = t - p_{n_O+k}$.

Identify Optimal Schedule

Find schedule σ_{k^*} such that $k^* = \arg \min_{0 \leq k \leq j} \{L_{\max}(\sigma_k) + \mu k\}$.

The purpose of the Preprocessing step of Algorithm L + MSD is to allow $L_{\max}(\sigma_{k-1})$ to be evaluated efficiently in the Schedule Construction and Evaluation step. This maximum lateness is computed by considering the maximum lateness $\bar{l}_k$ of all jobs that appear before $n_O + k$ in schedule σ_k , the maximum lateness L of those jobs of J_O that appear after $n_O + k$ in σ_k , and the maximum lateness $\bar{l}$ of jobs $n_O + k, \dots, n_O + n_N$ that appear at the end of σ_{k-1} .

THEOREM 9. *Algorithm L + MSD finds an optimal schedule for problem $1 \| L_{\max} + \mu D_{\max}(\pi^*)$ in $O(n + n_N \log n_N)$ time.*

PROOF. Algorithm L + MSD computes $\min_{0 \leq k \leq j} \{L_{\max}(\sigma_k) + \mu k\}$, where σ_k is found from Algorithm LMSD by solving problem $1 \| D_{\max}(\pi^*) \leq k \| L_{\max}$, and hence generates an optimal schedule because all possible values of k are considered.

We now consider the time complexity of Algorithm L + MSD. The Indexing step requires $O(n_N \log n_N)$ time to sequence the jobs of J_N in EDD order, and $O(n)$ time to construct an EDD sequence of all jobs. The Preprocessing step requires $O(n)$ time. Finally, the Schedule Construction and Evaluation step is performed at most j times, where $j \leq n_N$, and each application requires constant time. Thus, the overall time complexity of Algorithm L + MSD is $O(n + n_N \log n_N)$. $\square$

The computational complexity of problem $1 \| L_{\max} + \mu \sum D_j(\pi^*)$ remains open. Recall that the corresponding

constrained problem $1 \| \sum D_j(\pi^*) \leq k \| L_{\max}$ also has open complexity. We can show that these two problems are equivalent with respect to polynomial time solvability.

We next consider problem $1 \| L_{\max} + \mu \Delta_{\max}(\pi^*)$. Let k^* denote the maximum time disruption of an optimal schedule. An optimal schedule for problem $1 \| L_{\max} + \mu \Delta_{\max}(\pi^*)$ is then found by solving problem $1 \| \Delta_{\max}(\pi^*) \leq k^* \| L_{\max}$. If the jobs of J_N are indexed in EDD order, then the analysis in the proof of Theorem 2 shows that an optimal schedule σ^* has jobs $1, \dots, n_O$ interwoven with jobs $n_O + 1, \dots, n_O + i$ in overall EDD order, followed by the remaining jobs $n_O + i + 1, \dots, n_O + n_N$ in EDD order, for some index i where $0 \leq i \leq n_N$. Moreover, $k^* = C_{n_O}(\sigma^*) - C_{n_O}(\pi^*)$; otherwise, k^* could be reduced without affecting $L_{\max}(\sigma^*)$, thereby yielding a smaller value of $L_{\max} + \mu \Delta_{\max}(\pi^*)$ and contradicting the original choice of k^* . Thus, in an optimal schedule, the maximum time disruption is equal to the time disruption of job n_O . This discussion motivates the following algorithm.

Algorithm L + MTD

This algorithm is similar to Algorithm L + MSD, with the following changes and additions.

Preprocessing

Additionally, set $C_{n_O}(\sigma_j) = t$ after setting t .

Schedule Construction and Evaluation

Additionally, set $C_{n_O}(\sigma_{k-1}) = t$ after updating t .

Identify Optimal Schedule

Find schedule σ_{h^*} such that $h^* = \arg \min_{0 \leq k \leq j} \{L_{\max}(\sigma_k) + \mu(C_{n_O}(\sigma_k) - C_{n_O}(\pi^*))\}$.

THEOREM 10. *Algorithm L + MTD finds an optimal schedule for problem $1 \| L_{\max} + \mu \Delta_{\max}(\pi^*)$ in $O(n + n_N \log n_N)$ time.*

PROOF. The proof is similar to that of Theorem 9. $\square$

Finally, we consider problem $1 \| L_{\max} + \mu \sum \Delta_j(\pi^*)$.

THEOREM 11. *The recognition version of problem $1 \| L_{\max} + \mu \sum \Delta_j(\pi^*)$ is unary NP-complete.*

PROOF. The proof is by reduction from the 3-Partition Problem, as defined in the proof of Theorem 3. We use the instance of problem $1 \| \sum \Delta_j(\pi^*) \leq k \| L_{\max}$ defined there, with the modified threshold value $C = (2t^3y + t^2 + t)/(2t^3y + t^2 + t + 1)$ and the addition $\mu = 2/(2t^3y + t^2 + t + 1)$. Since $\sum_{j \in J} p_j = 2t^2y + t = \max_{j \in J} \{d_j\}$, any feasible schedule has $L_{\max} \geq 0$. Since $C < 1$ and $L_{\max}$ is integer because all the processing times and due dates are integer, we must have $L_{\max} = 0$ for any schedule with $L_{\max} + \mu \sum_{j \in J_O} \Delta_j(\pi^*) \leq C$. Thus, the condition $L_{\max} + \mu \sum_{j \in J_O} \Delta_j(\pi^*) \leq C$ is equivalent to $2 \sum_{j \in J_O} \Delta_j(\pi^*) / (2t^3y + t^2 + t + 1) \leq (2t^3y + t^2 + t) / (2t^3y + t^2 + t + 1)$, which can be expressed as $\sum_{j \in J_O} \Delta_j(\pi^*) \leq t^3y + t(t + 1)/2 = k$. Consequently, the recognition version of this instance of problem $1 \| L_{\max} + \mu \sum \Delta_j(\pi^*)$ has

a solution if and only if the recognition version of problem $1 \parallel \sum \Delta_j(\pi^*) \leq k \mid L_{\max}$ does so. The result now follows from Theorem 3. $\square$

Turning to problems with $\sum C_j$ scheduling cost, we first consider problem $1 \parallel \sum C_j + \mu D_{\max}(\pi^*)$. We propose the following algorithm that is analogous to Algorithm L + MSD.

Algorithm C + MSD

Input

Given p_j for $j = 1, \dots, n$, μ and π^* .

Indexing

Index the jobs of J_N in SPT order and sequence all n jobs in SPT order.

Preprocessing

If job n_O is sequenced before job $n_O + 1$, then terminate. Define j such that $n_O + j$ is the index of the last job of J_N that is sequenced before job n_O .

Compute the total processing time $\tilde{p}_i$ of all jobs of J_O sequenced between jobs $n_O + i$ and $n_O + i + 1$, and let $\tilde{w}_i$ be the number of those jobs for $i = 1, \dots, j - 1$.

Compute the total processing time $\tilde{P}$ of all jobs sequenced after $n_O + j$ up to and including job n_O , and let $\tilde{W}$ be the number of those jobs.

Set σ_j to be the schedule defined by the current job sequence, and compute $\sum_{h \in J} C_h(\sigma_j)$.

Schedule Construction and Evaluation

Execute the following for $k = j, \dots, 1$.

Form schedule σ_{k-1} from σ_k by moving job $n_O + k$ immediately after job n_O .

Compute $\sum_{h \in J} C_h(\sigma_{k-1}) = \sum_{h \in J} C_h(\sigma_k) + \tilde{P} - p_{n_O+k} \tilde{W}$.

If $k > 1$, then set $\tilde{P} = \tilde{P} + \tilde{p}_{k-1}$ and $\tilde{W} = \tilde{W} + \tilde{w}_{k-1}$.

Identify Optimal Schedule

Find schedule σ_{k^*} such that $k^* = \arg \min_{0 \leq k \leq j} \{\sum_{h \in J} C_h(\sigma_k) + \mu k\}$.

In Algorithm C + MSD, the total completion time for schedule σ_{k-1} is computed from the total completion time for schedule σ_k in the Schedule Construction and Evaluation step by observing that job $n_O + k$ is interchanged with a subsequence of jobs of J_O . The total processing time and the number of these jobs are stored as $\tilde{P}$ and $\tilde{W}$, respectively.

THEOREM 12. *Algorithm C + MSD finds an optimal schedule for problem $1 \parallel \sum C_j + \mu D_{\max}(\pi^*)$ in $O(n + n_N \log n_N)$ time.*

PROOF. The proof is similar to that of Theorem 9. $\square$

We next consider problem $1 \parallel \sum C_j + \mu \sum D_j(\pi^*)$. Recall from Lemma 3 that the (SPT, SPT) property holds. Assume that the jobs of J_N are indexed in SPT order. Let $\bar{\pi}^*$ denote the no-idle schedule for the jobs of J_N defined by the sequence $(n_O + 1, \dots, n_O + n_N)$, and let $C_j(\bar{\pi}^*)$ be

the completion time of job j in schedule $\bar{\pi}^*$ for $j = n_O + 1, \dots, n_O + n_N$. To solve problem $1 \parallel \sum C_j + \mu \sum D_j(\pi^*)$, we merge the two sequences used in schedules π^* and $\bar{\pi}^*$, basing decisions on the additional cost incurred by changing the initial completion times $C_1(\pi^*), \dots, C_{n_O}(\pi^*)$ and $C_{n_O+1}(\bar{\pi}^*), \dots, C_{n_O+n_N}(\bar{\pi}^*)$.

In any given schedule for problem $1 \parallel \sum C_j + \mu \sum D_j(\pi^*)$, suppose that i_j defines the relative position of job $n_O + j$ with respect to π^* for $j = 1, \dots, n_N$, where $i_j = 0$ or $i_j \in J_O$, so that unless $i_j = 0$, the relative position is a job of J_O . Specifically, the relative position of job $n_O + j$ is 0 if it is sequenced before job 1, the relative position of job $n_O + j$ is i if it is sequenced between jobs i and $i + 1$ for $i = 1, \dots, n_O - 1$, and the relative position of job $n_O + j$ is n_O if it is sequenced after job n_O . Let S_i be the subset of jobs of J_N with relative position i for $i = 0, \dots, n_O$. For the no-idle schedule defined by such a sequence, the completion time of job i is $C_i(\pi^*)$ plus the total processing time of jobs in $S_0, \dots, S_{i-1}$ that appear before job i for $i = 1, \dots, n_O$. Also, the completion time of job $n_O + j$ is $C_{n_O+j}(\bar{\pi}^*) + C_{i_j}(\pi^*)$ for $j = 1, \dots, n_N$, where $C_{i_j}(\pi^*)$ is the total processing time of jobs $1, \dots, i_j$ that appear before job $n_O + j$, and $C_0(\pi^*) = 0$. For the no-idle schedule defined by such a sequence,

$$\begin{aligned} \sum_{j \in J} C_j &= \sum_{i=1}^{n_O} \left(C_i(\pi^*) + \sum_{h=0}^{i-1} \sum_{j \in S_h} p_{n_O+j} \right) \\ &\quad + \sum_{j=1}^{n_N} (C_{n_O+j}(\bar{\pi}^*) + C_{i_j}(\pi^*)) \\ &= \sum_{i=1}^{n_O} C_i(\pi^*) + \sum_{j=1}^{n_N} C_{n_O+j}(\bar{\pi}^*) \\ &\quad + \sum_{i=0}^{n_O-1} (n_O - i) \sum_{j \in S_i} p_{n_O+j} + \sum_{j=1}^{n_N} C_{i_j}(\pi^*), \end{aligned} \quad (1)$$

$$\sum_{i \in J_O} D_i = \sum_{i=1}^{n_O} \sum_{h=0}^{i-1} |S_h| = \sum_{i=0}^{n_O-1} (n_O - i) |S_i|. \quad (2)$$

Because the first two terms in (1) are independent of the relative positions selected for the jobs of J_N , we do not consider them when merging the jobs of J_O and J_N . The cost of assigning job $n_O + j$ to relative position i is therefore

$$c_{ij} = C_i(\pi^*) + (p_{n_O+j} + \mu)(n_O - i), \quad i = 0, \dots, n_O, \quad j = 1, \dots, n_N. \quad (3)$$

Equation (1) defines a scheduling cost contribution $C_i(\pi^*) + (n_O - i)p_{n_O+j}$, when job $n_O + j$ is included in S_i , and Equation (2) similarly provides a disruption cost contribution $\mu(n_O - i)$.

Before describing an algorithm for problem $1 \parallel \sum C_j + \mu \sum D_j(\pi^*)$, we first establish a preliminary result about the values c_{ij} .

LEMMA 4. For any j , the values c_{ij} given by (3) for $i = 0, \dots, n_o$ are unimodal (but not necessarily strictly).

PROOF. For $1 \leq i \leq n_o - 1$, we use Equation (3) to evaluate

$$\begin{aligned} c_{i+1,j} - 2c_{ij} + c_{i-1,j} &= (C_{i+1}(\pi^*) - C_i(\pi^*)) \\ &\quad - (C_i(\pi^*) - C_{i-1}(\pi^*)) \\ &= p_{i+1} - p_i \geq 0, \end{aligned}$$

where the inequality $p_i \leq p_{i+1}$ is obtained from the fact that π^* is defined by an SPT sequence. This establishes the desired property. $\square$

We propose the following algorithm for problem $1 \parallel \sum C_j + \mu \sum D_j(\pi^*)$.

Algorithm C + TSD

Input

Given p_j for $j = 1, \dots, n$, μ and π^* .

Indexing

Index the jobs of J_N in SPT order. Set $C_0(\pi^*) = 0$.

Schedule Construction

Execute the following for $j = 1, \dots, n_N$.

Find $i_j = \arg \min_{i_{j-1} \leq i \leq n_o} \{C_i(\pi^*) + (p_{n_o+j} + \mu)(n_o - i)\}$
by bisection search, where $i_0 = 0$, and insert job $n_o + j$ in relative position i_j , so that it is sequenced last if $i_j = n_o$ and is sequenced immediately before job $i_j + 1$ otherwise.

THEOREM 13. Algorithm C + TSD finds an optimal schedule for problem $1 \parallel \sum C_j + \mu \sum D_j(\pi^*)$ in $O(n_N \log n)$ time.

PROOF. Lemma 3 establishes the (SPT, SPT) property. Given this property, Equations (1) and (2) show that the problem is equivalent to minimizing $c_{i_1,1} + \dots + c_{i_{n_N},n_N}$, subject to $i_1 \leq \dots \leq i_{n_N}$. In Algorithm C + TSD, we impose this constraint by solving $\min_{i_{j-1} \leq i \leq n_o} \{c_{ij}\}$ for $j = 1, \dots, n_N$. The optimality of this approach is established by using induction on the index j to show that

$$\min_{i_{j-1} \leq i \leq n_o} \{c_{ij}\} \leq \min_{0 \leq i < i_{j-1}} \{c_{ij}\} \quad (4)$$

for $j = 1, \dots, n_N$. Clearly, (4) holds for $j = 1$ because $i_0 = 0$. Suppose that (4) holds for $j = 1, \dots, j' - 1$, where $1 < j' \leq n_N$. Using (4) for $j = j' - 1$ and the definition of $i_{j'-1}$, we obtain $c_{i_{j'-1},j'-1} \leq c_{i,j'-1}$ for $i = 0, \dots, i_{j'-1} - 1$. Using (3), this inequality can be expressed as

$$\begin{aligned} C_{i_{j'-1}}(\pi^*) + (p_{n_o+j'-1} + \mu)(n_o - i_{j'-1}) \\ \leq C_i(\pi^*) + (p_{n_o+j'-1} + \mu)(n_o - i). \end{aligned}$$

The SPT indexing of the jobs of J_N implies that $p_{n_o+j'-1} \leq p_{n_o+j'}$. Therefore, because $i < i_{j'-1}$,

$$\begin{aligned} C_{i_{j'-1}}(\pi^*) + (p_{n_o+j'} + \mu)(n_o - i_{j'-1}) \\ \leq C_i(\pi^*) + (p_{n_o+j'} + \mu)(n_o - i), \end{aligned}$$

which shows that $c_{i_{j'-1},j'} \leq c_{i,j'}$ for $i = 0, \dots, i_{j'-1} - 1$, and therefore (4) holds for $j = j'$. We have now established the induction hypothesis, which is sufficient to prove the optimality of Algorithm C + TSD.

We now consider the time complexity of Algorithm C + TSD. The Indexing step requires $O(n_N \log n_N)$ time. The loop in the Schedule Construction step is performed n_N times, with each application requiring $O(\log n_o)$ time. Thus, the overall time complexity of Algorithm C + TSD is $O(n_N \log n)$. $\square$

We next consider problem $1 \parallel \sum C_j + \mu \Delta_{\max}(\pi^*)$, for which Lemma 3 shows that the (SPT, SPT) property holds. Recall that Algorithm L + MTD is developed from Algorithm L + MSD by noting that if σ^* is an optimal schedule, then the maximum time disruption is $k = C_{n_o}(\sigma^*) - C_{n_o}(\pi^*)$, and an optimal schedule is found by repeatedly solving problem $1 \parallel \Delta_{\max}(\pi^*) \leq k \mid L_{\max}$. We use similar arguments to establish an algorithm for problem $1 \parallel \sum C_j + \mu \Delta_{\max}(\pi^*)$ from Algorithm C + MSD.

Algorithm C + MTD

This algorithm is similar to Algorithm C + MSD, with the following changes and additions.

Preprocessing

Additionally, compute $C_{n_o}(\sigma_j)$.

Schedule Construction and Evaluation

Additionally, set $C_{n_o}(\sigma_{k-1}) = C_{n_o}(\sigma_k) - p_{n_o+k}$.

Identify Optimal Schedule

Find schedule σ_{k^*} such that $k^* = \arg \min_{0 \leq k \leq j} \{\sum_{h \in J} C_h(\sigma_k) + \mu(C_{n_o}(\sigma_k) - C_{n_o}(\pi^*))\}$.

THEOREM 14. Algorithm C + MTD finds an optimal schedule for problem $1 \parallel \sum C_j + \mu \Delta_{\max}(\pi^*)$ in $O(n + n_N \log n_N)$ time.

PROOF. The proof is similar to the proof of Theorem 9. $\square$

Finally, we consider problem $1 \parallel \sum C_j + \mu \sum \Delta_j(\pi^*)$, for which we develop an algorithm that is similar to Algorithm C + TSD. Again, Lemma 3 shows that the (SPT, SPT) property holds, and as before let the jobs of J_N be indexed in SPT order. Also, in any given schedule, suppose that i_j defines the relative position of job $n_o + j$ with respect to π^* for $j = 1, \dots, n_N$, and S_i is the set of jobs of J_N with relative position i for $i = 0, \dots, n_o$. Similar arguments to those used in the derivation of Equation (2) show that

$$\sum_{i \in J_o} \Delta_i = \sum_{i=1}^{n_o} \sum_{h=0}^{i-1} \sum_{j \in S_h} p_j = \sum_{i=0}^{n_o-1} (n_o - i) \sum_{j \in S_i} p_j. \quad (5)$$

Further, Equations (1) and (5) show that the cost of assigning job $n_o + j$ to relative position i in problem $1 \parallel \sum C_j + \mu \sum \Delta_j(\pi^*)$ is given by

$$\begin{aligned} c_{ij} &= C_i(\pi^*) + p_{n_o+j}(1 + \mu)(n_o - i), \\ &\quad i = 0, \dots, n_o, j = 1, \dots, n_N, \quad (6) \end{aligned}$$

where $C_0(\pi^*) = 0$. Thus, we have the following algorithm for problem $1 \parallel \sum C_j + \mu \sum \Delta_j(\pi^*)$.

Algorithm C + TTD

This algorithm is similar to Algorithm C + TSD, with the following alternative definition of i_j in the Schedule Construction step:

$$i_j = \arg \min_{i_{j-1} \leq i \leq n_O} \{C_i(\pi^*) + p_{n_O+j}(1 + \mu)(n_O - i)\}.$$

THEOREM 15. *Algorithm C + TTD finds an optimal schedule for problem 1 $\parallel \sum C_j + \mu \sum \Delta_j(\pi^*)$ in $O(n_N \log n)$ time.*

PROOF. A result analogous to Lemma 4 can be established for this problem. The proof is then similar to that of Theorem 13. $\square$

6. Multiple-Disruption Problems

As discussed in §1, the problems studied in §§4 and 5 consider a single disruption that occurs after the planning of an optimal schedule. However, if the frequency of new job arrivals is relatively high, then the approximation used by Gershwin (1987) is less accurate, and we need to consider multiple disruptions. It is important to note that as a result of previous disruptions, the original schedule, denoted by π , may no longer be optimal with respect to the scheduling cost. Indeed, we allow the original schedule to be arbitrary, possibly containing machine idle time.

We begin our discussion with a result that compares the difficulty of single- and multiple-disruption problems.

LEMMA 5. *Consider P_S , a problem with a single disruption, and P_M , an equivalent problem with multiple disruptions.*

(a) *An optimal algorithm for problem P_M also solves problem P_S .*

(b) *If the recognition version of problem P_S is binary (respectively, unary) NP-complete, then the recognition version of problem P_M is also binary (respectively, unary) NP-complete.*

PROOF. We consider each part in turn.

(a) Because the algorithm for problem P_M requires as input only an arbitrary schedule of the original jobs, we can input the optimal schedule π^* .

(b) In a polynomial or pseudopolynomial time reduction for problem P_S , the constructed instance of that problem is also an instance of problem P_M . $\square$

To illustrate the differences in solvability that can arise from multiple disruptions, we consider problem 1 $\mid \Delta_{\max}(\pi^*) \leq k \mid L_{\max}$. Theorem 2 shows that this problem can be solved in $O(n + n_N \log n_N)$ time in the presence of a single disruption. However, the corresponding problem with multiple disruptions, 1 $\mid \Delta_{\max}(\pi) \leq k \mid L_{\max}$, is less tractable.

THEOREM 16. *The recognition version of problem 1 $\mid \Delta_{\max}(\pi) \leq k \mid L_{\max}$ is unary NP-complete.*

PROOF. Our proof is by reduction from the 3-Partition Problem, as defined in the proof of Theorem 3. Consider the following instance of the recognition version of problem 1 $\mid \Delta_{\max}(\pi) \leq k \mid L_{\max}$:

$$\begin{aligned} n_O &= 2t + 1 \\ n_N &= 3t \\ p_j &= y, \quad j = 1, \dots, 2t \\ d_j &= jy, \quad j = 1, 3, \dots, 2t - 1 \\ d_j &= 4ty + jy/2, \quad j = 2, 4, \dots, 2t \\ p_{2t+1} &= 2ty \\ d_{2t+1} &= 4ty \\ p_j &= a_{j-2t-1}, \quad j = 2t + 2, \dots, 5t + 1 \\ d_j &= 2ty, \quad j = 2t + 2, \dots, 5t + 1 \\ k &= 2ty \\ C &= 0, \end{aligned}$$

and π is the no-idle schedule defined by the sequence $(2t + 1, 1, 2, \dots, 2t)$.

We prove that there exists a feasible schedule for this instance of problem 1 $\mid \Delta_{\max}(\pi) \leq k \mid L_{\max}$ with $L_{\max} \leq C$ if and only if there exists a solution to the 3-Partition Problem.

($\Rightarrow$) Consider the no-idle schedule σ defined by the sequence $(1, 2t + 2, 2t + 3, 2t + 4, 3, 2t + 5, 2t + 6, 2t + 7, \dots, 2t - 1, 5t - 1, 5t, 5t + 1, 2t + 1, 2, 4, \dots, 2t)$. Then, we have $C_j(\pi) - C_j(\sigma) = 2ty = k$ for $j = 1, 3, \dots, 2t - 1$, and $C_{2t+1}(\sigma) - C_{2t+1}(\pi) = 2ty = k$. Further, $0 \leq C_j(\sigma) - C_j(\pi) \leq (2t - 1)y < k$ for $j = 2, 4, \dots, 2t$. Thus, $\Delta_{\max}(\pi, \sigma) = k$. Also, $C_j(\sigma) = d_j$ for $j = 1, \dots, 2t + 1$. Finally, $C_j(\sigma) \leq d_j$ for $j = 2t + 2, \dots, 5t + 1$. Therefore, $L_{\max}(\sigma) = 0 = C$.

($\Leftarrow$) Consider a schedule σ with $\Delta_{\max}(\pi, \sigma) \leq k$ and $L_{\max}(\sigma) \leq 0$. We first claim that $C_j(\sigma) = jy$ for $j = 1, 3, \dots, 2t - 1$. To show this, consider any job j , where $j \in \{1, 3, \dots, 2t - 1\}$. Because $C_j(\pi) = 2ty + jy$, if $C_j(\sigma) < jy$, then $C_j(\pi) - C_j(\sigma) > 2ty = k$. Alternatively, if $C_j(\sigma) > jy = d_j$, then $L_{\max} > 0$. Together, these observations establish our claim.

Jobs $2t + 2, \dots, 5t + 1$ are each completed by their common due date of $2ty$, which implies that they must be scheduled in the intervals $[y, 2y], [3y, 4y], \dots, [(2t - 3)y, (2t - 2)y], [(2t - 1)y, 2ty]$, during which the machine is not processing the jobs $1, 3, \dots, 2t - 1$. Because $\sum_{j=2t+2}^{5t+1} p_j = ty$, each of these intervals contains exactly y units of processing. From the definition of the 3-Partition Problem, $y/4 < p_j < y/2$ for $j = 2t + 2, \dots, 5t + 1$; therefore, exactly three jobs are scheduled in each interval, which establishes the existence of a solution to the 3-Partition Problem. $\square$

We next consider problem 1 $\mid \sum \Delta_j(\pi) \leq k \mid L_{\max}$.

COROLLARY 1. *The recognition version of problem 1 | $\sum \Delta_j(\pi) \leq k$ | $L_{\max}$ is unary NP-complete.*

PROOF. The result follows from Theorem 3 and Lemma 5. $\square$

However, multiple disruptions do not prevent the efficient solution of all rescheduling problems of the type we consider. Recall from Theorem 4 that problem 1 | $D_{\max}(\pi^*) \leq k$ | $\sum C_j$ can be solved in $O(n + n_N \log n_N)$ time in the presence of a single disruption. Although a more complicated solution procedure is needed for the corresponding problem with multiple disruptions, 1 | $D_{\max}(\pi) \leq k$ | $\sum C_j$, this problem can still be solved in polynomial time. Moreover, our algorithm easily extends to solve problem 1 || $\sum C_j + \mu D_{\max}(\pi)$.

THEOREM 17. *An optimal schedule for problem 1 | $D_{\max}(\pi) \leq k$ | $\sum C_j$ can be found in $O(n^3)$ time.*

PROOF. We show that the problem can be solved as a linear assignment problem. First, we note that the cost of a no-idle schedule defined by an arbitrary sequence σ is $\sum_{j \in J} C_j(\sigma) = \sum_{j=1}^n \sum_{i=1}^j p_{\sigma(i)} = \sum_{i=1}^n (n - i + 1) p_{\sigma(i)}$. Thus, the linear assignment problem has costs c_{ij} , where i denotes the position in the sequence to be constructed and j denotes the job, defined by

$$c_{ij} = \begin{cases} (n - i + 1)p_j & \text{for } j \in J_N \text{ and } j \in J_O \\ & \text{with } |\pi^{-1}(j) - i| \leq k \\ \infty & \text{otherwise} \end{cases}$$

for $i = 1, \dots, n$,

where $\pi^{-1}(j)$ is the position of job j in π . Solving a linear assignment problem requires $O(n^3)$ time (Kuhn 1955). $\square$

COROLLARY 2. *An optimal schedule for problem 1 || $\sum C_j + \mu D_{\max}(\pi)$ can be found in $O(n_N n^3)$ time.*

PROOF. For each possible value of k , where $k \in \{0, \dots, n_N\}$, problem 1 | $D_{\max}(\pi) \leq k$ | $\sum C_j$ is solved, and the best solution is selected. Theorem 17 shows that this algorithm requires $O(n_N n^3)$ time. $\square$

Although a comprehensive study of rescheduling problems with multiple disruptions is beyond the scope of this paper, the results above show that their solution may require insights and techniques that are different from those used in single-disruption problems.

7. Concluding Remarks

This paper studies the issue of rescheduling to allow for the unexpected arrival of new jobs, taking into account the effect of the disruption on a previously planned optimal schedule. The effect of disruption is measured either by the extent of repositioning within the processing sequence, or by the change in completion time, for the original jobs. We

consider two classical objectives for the scheduling cost, and allow the effect of disruption to be modeled either as a constraint or as an additional component in the objective. For most of the problems considered, we provide either an efficient solution procedure or an intractability result that demonstrates the probable nonexistence of such a procedure. Moreover, our solution procedures are sufficiently simple that they can be implemented on a spreadsheet for practical use. We also address the issue of how to respond to repeated disruptions from the arrival of new jobs by providing models that start from an arbitrary, rather than optimal, planned schedule.

Several important issues remain open for future research. First, it would be useful to resolve the computational complexity of the two problems discussed in this paper that remain open. Second, the situation where multiple disruptions occur needs to be investigated thoroughly. Third, it would also be valuable to extend our work to consider other classical definitions of scheduling cost. Fourth, there are other relevant measures of disruption effects, besides the four that we have considered, that also require examination. Fifth, our work should be extended to include other manufacturing environments such as flow shops and job shops. Finally, as discussed in the introduction, various disruptions besides the arrival of new jobs occur frequently in practice. For these disruptions, models need to be developed to investigate the trade-off between scheduling cost and disruption effects. We hope that our work will stimulate further development and implementation of rescheduling models to mitigate the effects of the disruptions that occur frequently in manufacturing practice.

Acknowledgments

This research was supported in part by the National Science Foundation (grant number DMI-9821033), by the Summer Fellowship Program (Fisher College of Business, The Ohio State University), by the CORMSIS Visitor Programme (University of Southampton), and by the Engineering and Physical Sciences Research Council (grant number GR/R97696/01).

References

- Bean, J. C., J. R. Birge, J. Mittenthal, C. E. Noon. 1991. Matchup scheduling with multiple resources, release dates and disruptions. *Oper. Res.* **39** 470–483.
- Brucker, P. 1998. *Scheduling Algorithms*, 2nd ed. Springer, Berlin, Germany.
- Chang, Y.-L., R. S. Sullivan, U. Bagchi. 1984. Experimental investigation of quasi-realtime scheduling on flexible manufacturing systems. *Proc. First ORSA/TIMS Special Interest Conf. on Flexible Manufacturing Systems*, Ann Arbor, MI, 307–312.
- Church, L. K., R. Uzsoy. 1992. Analysis of periodic and event-driven rescheduling policies in dynamic shops. *Internat. J. Comput.-Integrated Manufacturing* **5** 153–163.
- Conway, R. W., W. L. Maxwell, L. W. Miller. 1967. *Theory of Scheduling*. Addison-Wesley, Reading, MA.

- Daniels, R. L., P. Kouvelis. 1995. Robust scheduling to hedge against processing time uncertainty in single-stage production. *Management Sci.* **41** 363–376.
- Donath, M., R. J. Graves. 1985. Flexible assembly systems: Near real-time scheduling of multiple products. Technical report, Department of Industrial Engineering and Operations Research, University of Massachusetts, Amherst, MA.
- Garey, M. R., D. S. Johnson. 1979. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. Freeman, San Francisco, CA.
- Garey, M. R., R. E. Tarjan, G. T. Wilfong. 1988. One-processor scheduling with symmetric earliness and tardiness penalties. *Math. Oper. Res.* **13** 330–348.
- Gershwin, S. B. 1987. A hierarchical framework for discrete event scheduling in manufacturing systems. P. Varaiya, A. B. Kurzhanski, eds. *Lecture Notes in Control and Information Systems 103: Discrete Event Systems: Models and Applications*, Springer-Verlag, Berlin, Germany, 197–216.
- Graham, R. L., E. L. Lawler, J. K. Lenstra, A. H. G. Rinnooy Kan. 1979. Optimization and approximation in deterministic machine scheduling: A survey. *Ann. Discrete Math.* **5** 287–326.
- Jackson, J. R. 1955. Scheduling a production line to minimize maximum tardiness. Research report 43, Management Science Research Project, University of California, Los Angeles, CA.
- Jain, A. K., H. A. Elmaraghy. 1997. Production scheduling/rescheduling in flexible manufacturing. *Internat. J. Production Res.* **35** 281–309.
- Koulamas, C., G. J. Kyparisis. 2001. Single machine scheduling with release times, deadlines and tardiness objectives. *Eur. J. Oper. Res.* **133** 447–453.
- Kuhn, H. W. 1955. The Hungarian method for the assignment problem. *Naval Res. Logist. Quart.* **2** 83–97.
- Li, R., Y. Shyu, S. Adiga. 1993. A heuristic rescheduling algorithm for computer-based production scheduling systems. *Internat. J. Production Res.* **31** 1815–1826.
- Mehta, S. V., R. Uzsoy. 1999. Predictable scheduling of a single machine subject to breakdowns. *Internat. J. Comput.-Integrated Manufacturing* **12** 15–38.
- O'Donovan, R., R. Uzsoy, K. N. McKay. 1999. Predictable scheduling of a single machine with breakdowns and sensitive jobs. *Internat. J. Production Res.* **37** 4217–4233.
- Pinedo, M. 1995. *Scheduling: Theory, Algorithms and Systems*. Prentice Hall, Englewood Cliffs, NJ.
- Raman, N., F. B. Talbot, R. V. Rachamadugu. 1989. Due date based scheduling in a general flexible manufacturing system. *J. Oper. Management* **8** 115–132.
- Smith, W. E. 1956. Various optimizers for single-stage production. *Naval Res. Logist. Quart.* **3** 59–66.
- Unal, A. T., R. Uzsoy, A. S. Kiran. 1997. Rescheduling on a single machine with part-type dependent setup times and deadlines. *Ann. Oper. Res.* **70** 93–113.
- Wu, S. D., R. H. Storer, P.-C. Chang. 1992. A rescheduling procedure for manufacturing systems under random disruptions. G. Fandel, T. Gullledge, A. Jones, eds. *New Directions for Operations Research in Manufacturing*. Springer, Berlin, Germany, 292–308.
- Wu, S. D., R. H. Storer, P.-C. Chang. 1993. One-machine rescheduling heuristics with efficiency and stability as criteria. *Comput. Oper. Res.* **20** 1–14.
- Yellig, E. J., G. T. Mackulak. 1997. Robust deterministic scheduling in stochastic environments: The method of capacity hedge points. *Internat. J. Production Res.* **35** 369–379.